

Eventos de domínio do FestaLink

Este documento descreve os eventos publicados pelo backend, o canal usado, o formato do payload e o consumidor responsável. Os eventos materializam a comunicação assíncrona entre a parte síncrona da API REST e a parte que notifica o prestador em tempo real.

Canal

Todos os eventos trafegam por um único canal Redis configurável via variável `EVENTOS_CANAL`. O valor padrão é `festalink:eventos`. O canal é usado em modo pub/sub: o backend é o único produtor e mantém um consumer interno que repassa as mensagens ao notificador WebSocket.

Formato base do envelope

Todo evento publicado segue o mesmo envelope:

```
{
  "type": "NomeDoEvento",
  "timestamp": "2026-05-25T23:41:45.919Z",
  "payload": { ... }
}
```

Os fields `type` e `timestamp` são fixos. O `payload` varia conforme o evento.

Eventos

Evento	Produtor (use case)	Quando dispara	Canal	Consumidor interno	Efeito
ReservaSolicitada	CriarReserva	Após persistir uma reserva com status <code>pendente</code>	<code>festalink:eventos</code>	RedisEventConsumer → WebSocketNotifier	Push para o prestador dono do salão
ReservaAprovada	AprovarReserva	Após atualizar a reserva para <code>aprovada</code>	<code>festalink:eventos</code>	RedisEventConsumer → WebSocketNotifier	Push para o prestador dono do salão
ReservaRecusada	RecusarReserva	Após atualizar a reserva para <code>recusada</code>	<code>festalink:eventos</code>	RedisEventConsumer → WebSocketNotifier	Push para o prestador dono do salão

A publicação acontece sempre depois da transação principal, em microtarefa, fora do caminho da resposta HTTP. Uma falha de publicação é registrada no log mas não derruba a requisição que originou o evento.

Payloads

ReservaSolicitada

```
{
  "type": "ReservaSolicitada",
  "timestamp": "2026-05-25T23:41:45.919Z",
  "payload": {
    "reservaId": 4,
    "salaoId": 4,
    "clienteId": 6,
    "dataEvento": "2026-06-05",
    "horaInicio": "19:00",
    "horaFim": "22:00"
  }
}
```

ReservaAprovada

```
{
  "type": "ReservaAprovada",
  "timestamp": "2026-05-25T23:41:47.236Z",
  "payload": {
    "reservaId": 4,
    "salaoId": 4,
    "clienteId": 6
  }
}
```

ReservaRecusada

```
{
  "type": "ReservaRecusada",
  "timestamp": "2026-05-25T23:41:50.111Z",
  "payload": {
    "reservaId": 4,
    "salaoId": 4,
    "clienteId": 6
  }
}
```

Caminho pub → consume → push

1. O use case correspondente conclui sua transação e chama `eventPublisher.publicar(evento)`.
2. O `RedisEventPublisher` serializa o envelope como JSON e dispara `PUBLISH` no canal.
3. O `RedisEventConsumer`, inscrito no mesmo canal (`SUBSCRIBE`), recebe a mensagem via callback `on('message')`.
4. O consumer faz `JSON.parse` e chama `wsNotifier.notificarPrestadorDoSalao(salaoId, evento)`.
5. O notificador consulta o `SalaoRepository` para descobrir o prestador dono do salão e, se houver socket aberto, empurra o envelope serializado pelo WebSocket.

O fato de o produtor e o consumer estarem desacoplados pelo canal significa que substituir o pub/sub por outro broker (RabbitMQ, NATS) ou separar o consumer em outro processo é uma mudança contida no adapter, sem alterar nenhum use case.

WebSocket de notificação

O `WebSocketNotifier` abre um servidor WebSocket na porta configurada em `WS_PORT` (padrão 3001). A autenticação acontece no handshake: o cliente conecta passando o JWT na query string (`?token=<jwt>`). O notificador valida o token, exige `papel === 'prestador'` e indexa o socket pelo `prestadorId`.

Mensagens publicadas pelo backend para um prestador chegam exatamente no formato do envelope acima. Ao conectar, o cliente também recebe uma mensagem informativa `{"type": "Conectado", "prestadorId": N}` para confirmar a sessão.